SEG3502 – Lab 8
Services Web GraphQL

L'objectif de ce laboratoire est de présenter le développement de services Web basés sur GraphQL pour l'accès aux données. Nous allons (1) développer une API de service Web GraphQL pour un modèle de données supporté par une base de données NoSQL et (2) créer une application cliente Angular qui consomme l'API.

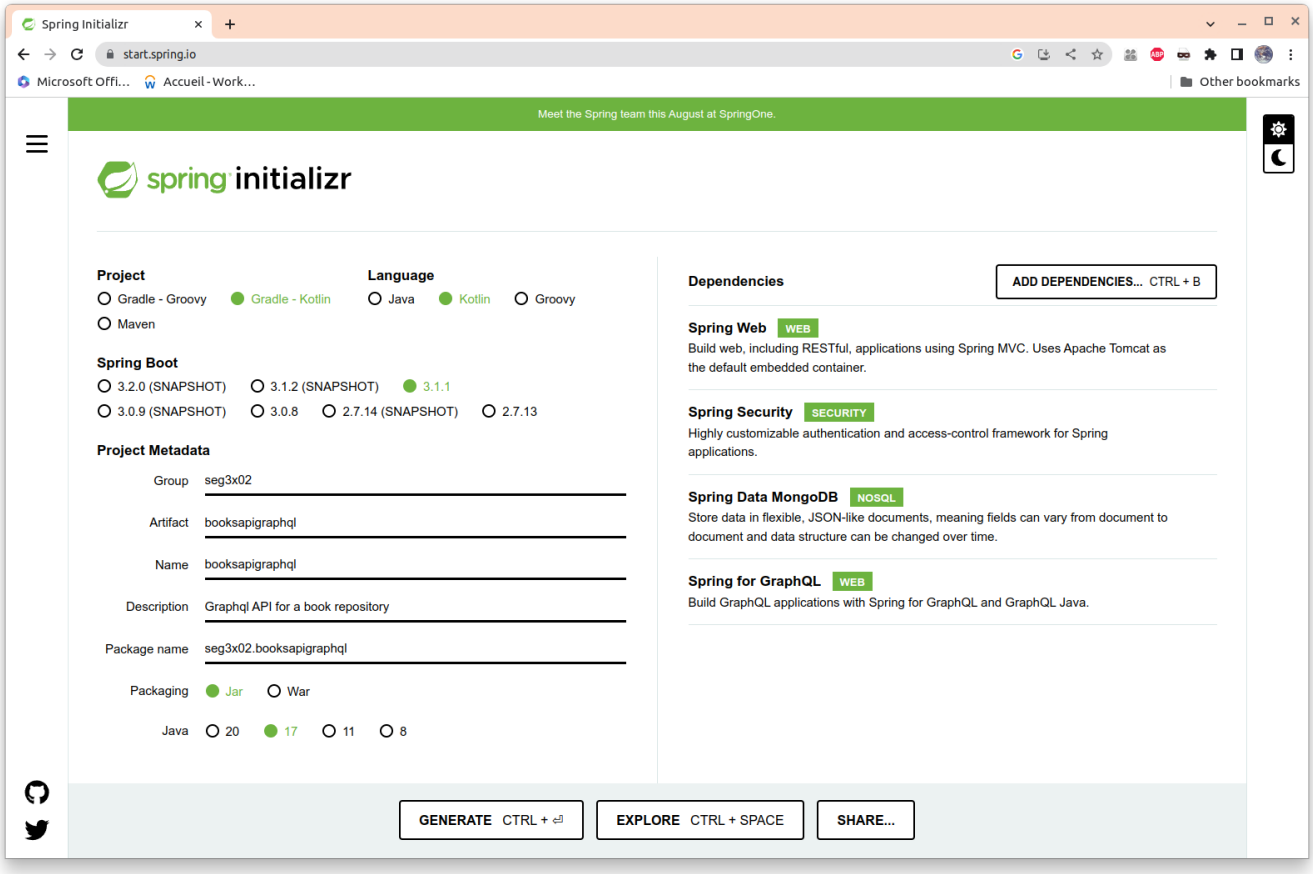
Le code source du laboratoire est disponible à https://github.com/stephanesome/graphQLService.git. Une application serveur SpringBoot peut être trouvée dans la branche *server*, tandis qu'une application cliente Angular peut être trouvée dans la branche *client*.

Serveur GraphQL

Nous développerons un serveur GraphQL pour la librairie développé dans le cadre du Lab 3. Nous stockerons les données dans un magasin de données MongoDB.

Configuration de l'application

Configurez un projet Spring Boot à l'aide de Spring Intializr.



Sélectionnez les dépendances suivantes : Spring Web, Spring Data MongoDB, Spring for GraphQL et Spring Security. Spring for GraphQL (https://spring.io/projects/spring-graphql) fournit du support

pour GraphQL dans Spring. Spring Web est nécessaire pour exposer l'API sur HTTP, Spring Data MongoDB fournit le nécessaire pour interagir avec le magasin de données orienté documents NoSQL MongoDB et Spring Security est nécessaire pour configurer l'accès de sécurité à l'API.

MongoDB

Nous utilisons MongoDB (https://www.mongodb.com/) comme magasin de données pour conserver nos entités. Vous pouvez télécharger et installer l'édition communautaire directement sur votre ordinateur (https://docs.mongodb.com/manual/installation/). Cependant, nous utiliserons Docker pour une installation plus simple et plus pratique.

Exécutez la commande suivante pour télécharger une image Docker et configurer MongoDB pour cette application.

```
docker run -d -p 27017:27017 --name mongodb -d mongo
```

Propriétés Spring

Spécifiez les propriétés de l'application dans `src/main/resources/application.properties` comme suit.

```
1. server.port=9000
2. spring.data.mongodb.database=books-mongo
3. spring.data.mongodb.port=27017
4. spring.graphql.graphiql.enabled=true
```

Nous avons spécifié un port pour l'application, le nom du magasin de données MongoDB et son port d'accès. Nous avons également activé l'outil GraphiQL (https://github.com/graphql/graphiql) qui fournit une interface utilisateur Web pour interagir avec les serveurs GraphQL. Ceci est utile pour vérifier qu'un serveur fonctionne correctement avant de connecter des clients.

Entités

Créez un package `seg3x02.booksapigraphql.entity`. Créez une classe de données Kotlin Book dans ce package puis éditez comme suit.

```
1. package seg3x02.booksapigraphql.entity
2.
3. import org.springframework.data.annotation.Id
4. import org.springframework.data.mongodb.core.mapping.Document
5.
6. @Document(collection = "book")
7. data class Book(var bookNumber: Int,
8.                 var category: String,
9.                 var title: String,
10.                var cost: Float,
11.                var year: String?,
12.                var description: String?) {
```

```
13.     @Id
14.     var bookId: String = ""
15.
16.     @Transient
17.     var authors: List<Author> = ArrayList()
18. }
```

La classe est annotée @Document, une annotation Spring Data qui identifie une classe comme une entité persistée en tant que document dans MongoDB.

De même, créez la classe de données Author et modifiez-la comme suit.

```
1.  package seg3x02.booksapigraphql.entity
2.
3.  import org.springframework.data.mongodb.core.mapping.Document
4.
5.  @Document(collection = "author")
6.  data class Author(var bookNumber: Int,
7.                    var firstName: String,
8.                    var lastName: String)
```

Repositories

Nous créons des référentiels (repository) pour accéder au magasin de données. Créez un package `seg3x02.booksapigraphql.repository`. Créez une interface Kotlin BookRepository dans le package et modifiez-le comme suit.

```
1.  package seg3x02.booksapigraphql.repository
2.
3.  import org.springframework.data.mongodb.repository.MongoRepository
4.  import org.springframework.stereotype.Repository
5.  import seg3x02.booksapigraphql.entity.Book
6.
7.  @Repository
8.  interface BookRepository: MongoRepository<Book, String>
```

De même, créez une interface Kotlin AuthorRepository pour Author et modifiez comme suit.

```
1.  package seg3x02.booksapigraphql.repository
2.
3.  import org.springframework.data.mongodb.repository.MongoRepository
4.  import org.springframework.stereotype.Repository
5.  import seg3x02.booksapigraphql.entity.Author
6.
7.  @Repository
8.  interface AuthorRepository: MongoRepository<Author, Int>
```

Schéma GraphQL

Un schéma GraphQL spécifie l'API exposée par un serveur GraphQL. Il décrit les données pouvant être requises, les requêtes supportées et les modifications autorisées sur les données.

Créez un fichier nommé `book.graphqls` dans le dossier `src/main/resources/graphql` et modifiez-le comme suit.

```
1.  type Query {
2.      books: [Book]
3.      bookById(bookId: ID!): Book
4.      bookByNumber(bookNumber: Int!): Book
5.  }
6.
7.  type Book {
8.      bookId: ID!
9.      bookNumber: Int!
10.     category: String!
11.     title: String!
12.     cost: Float!
13.     year: String
14.     description: String
15.     authors: [Author]
16. }
17.
18. input CreateBookInput {
19.     bookNumber: Int,
20.     category: String,
21.     title: String,
22.     cost: Float,
23.     year: String,
24.     description: String
25. }
26.
27. type Mutation {
28.     newBook(createBookInput: CreateBookInput!) : Book!
29.     deleteBook(bookId: ID!) : Boolean
30.     updateBook(bookId: ID!, createBookInput: CreateBookInput!) : Book!
31. }
```

Un schéma GraphQL (https://graphql.org/learn/) se compose de définitions de types. Chaque type a un ou plusieurs champs. Un champ peut prendre zéro ou plusieurs arguments et retourner une valeur d'un type spécifique.

Le type Query définit le point d'entrée de chaque requête GraphQL. Nous spécifions trois de ces requêtes en tant que champs du type (lignes 1-5). Une requête pour récupérer une liste d'instances de

type Book, une requête qui prend en paramètre un ID et retourne une instance de Book et une requête qui prend un Int et retourne une instance de Book.

Le type Book spécifie les objets Book qui peuvent être obtenus à partir du service. Les champs avec '!' ajouté à leur type ne sont *pas nullables*. Le service retourne toujours une valeur lorsque de tels champs sont interrogés. GraphQL fournit un ensemble de types prédéfinis (tels que : Int, Float, String, ...) qui sont des types scalaires correspondant aux feuilles des requêtes. Un champ peut également être typé avec un type d'objet comme pour le champ authors (ligne 15).

Une mutation est un type spécial de requête qui modifie les données. Le type Mutation définit le point d'entrée de chaque mutation. Le schéma définit trois mutations : newBook pour ajouter un nouveau Book, deleteBook pour supprimer un Book et updateBook pour mettre à jour les informations d'un Book.

Créez un nom de fichier `author.graphqls` dans le dossier `src/main/resources` et modifiez-le comme suit pour ajouter un schéma GraphQL pour Author.

```
1.  extend type Query {
2.      authors(bookNumber: Int!): [Author]
3.  }
4.
5.  type Author { bookNumber: Int!
6.      firstName: String!
7.      lastName: String!
8.  }
9.
10. extend type Mutation {
11.     newAuthor(bookNumber: Int!
12.         firstName: String!,
13.         lastName:String!) : Author!
14. }
```

Le schéma étend les types de requête et de mutation pour Author. Notez que cela aurait pu être défini dans le schéma principal. Nous avons séparés pour une meilleure modularité.

Résolveurs

Un resolver (résolveur) spécifie la logique applicative pour traiter un champ dans le cadre de l'exécution de requêtes et de mutations. Les résolveurs sont définis comme des fonctions. Spring pour GraphQL permet de spécifier les fonctions résolveurs dans des contrôleurs Web en utilisant des annotations.

Créez un package `seg3x02.booksapigraphql.resolvers`. Créez une classe Kotlin BooksController dans le package et modifiez-la en fonction du code source sur GitHub. Nous ne présentons ici qu'un extrait à titre d'illustration.

```
1.  @Controller
2.  class BooksController(private val bookRepository: BookRepository,
3.                        private val mongoOperations: MongoOperations
4.  ) {
5.      @QueryMapping
6.      fun books(): List<Book> {
7.          return bookRepository.findAll()
8.      }
9.
10.     @SchemaMapping(typeName="Book", field="authors")
11.     fun authors(book: Book): List<Author> {
12.         val query = Query()
13.         query.addCriteria(Criteria.where("bookNumber").`is`(book.bookNumber))
14.         return mongoOperations.find(query, Author::class.java)
15.     }
16.     ...
17.    @MutationMapping
18.     fun newBook(@Argument("createBookInput") input: CreateBookInput) : Book {
19.         if (input.bookNumber != null &&
20.                 input.category != null &&
21.                 input.title != null && input.cost != null) {
22.             val book =
23.               Book(input.bookNumber, input.category, input.title, input.cost, input.year, input.description)
24.             book.bookId = UUID.randomUUID().toString()
25.             bookRepository.save(book)
26.             return book
27.         } else {
28.             throw Exception("Invalid input")
29.         }
30.     }
31.     …
32. }
```

Les résolveurs sont définis comme des fonctions annotées. L'annotation @QueryMapping est utilisée pour résoudre les champs du type Query, l'annotation @MutationMapping pour les champs de type Mutation et l'annotation @SubscriptionMapping pour les champs de type Subscription. L'annotation générique @SchemaMapping peut être utilisée pour définir un résolveur pour un champ quel que soit son type en spécifiant des valeurs d'attributs.

La fonction books (15-8) correspond au champs books du type Query. Les données pour le champ authors du Book sont mappées par la fonction authors. De même, la fonction newBook (17-30) mappe le champ newBook déclarée dans le type Mutation.

Créez une classe Kotlin AuthorController dans le package `seg3x02.booksapigraphql.resolvers` et modifiez-la en fonction du code source sur GitHub. Cette classe spécifie les résolveurs pour le schéma author.

Nous devons également créer des classes qui mappent les types d'entrée spécifiés dans les schémas GraphQL. Créez un sous-package `types` dans le package `seg3x02.booksapigraphql.resolvers`. Créez une classe Kotlin CreateBookInput et modifiez-la comme suit :

```
1.  package seg3x02.booksapigraphql.resolvers.types
2.
3.  class CreateBookInput(
4.      val bookNumber: Int? = null,
5.      val description: String? = null,
6.      val year: String? = null,
7.      val cost: Float? = null,
8.      val title: String? = null,
9.      val category: String? = null
10. )
```

De même pour la mutation newAuthor, créez une classe Kotlin CreateAuthorInput et modifiez-la comme suit :

```
1.  package seg3x02.booksapigraphql.resolvers.types
2.
3.  class CreateAuthorInput(
4.      val bookNumber: Int = 0,
5.      val firstName: String = "",
6.      val lastName: String = "")
```

Configuration Web

Nous ajoutons de la configuration de sécurité pour permettre l'accès client au serveur.

Créez une classe WebSecurityConfig dans le package `seg3x02.booksapigraphql.configuration` et modifiez-la comme suit.

```
1.  package seg3x02.booksapigraphql.configuration
2.
3.  import org.springframework.context.annotation.Bean
4.  import org.springframework.context.annotation.Configuration
5.  import org.springframework.security.config.Customizer
6.  import org.springframework.security.config.annotation.web.builders.HttpSecurity
7.  import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
8.  import org.springframework.security.web.SecurityFilterChain
9.
```

```
10. @Configuration
11. @EnableWebSecurity
12. class WebSecurityConfig  {
13.     @Bean
14.     fun filterChain(http: HttpSecurity): SecurityFilterChain {
15.         http.cors(Customizer.withDefaults())
16.         http.csrf(({ csrf -> csrf.disable() }))
17.         http.authorizeHttpRequests{auth -> auth.anyRequest().permitAll()}
18.         return http.build()
19.     }
20. }
```

Spring Security nous permet de configurer la sécurité d'application dans une classe annotée @EnableWebSecurity. La configuration est spécifiée comme un Bean. Notre configuration active le Cross-Origin Resource Sharing (CORS) sur la ligne 15, nous désactivons la vérification Cross-Site Request Forgery (CSRF) à la ligne 16 car l'application est une API Web, et autorisons toutes les requêtes sans authentification (ligne 17).

Créez une classe WebConfiguration dans le package `seg3x02.booksapigraphql.configuration` et modifiez-la comme suit.

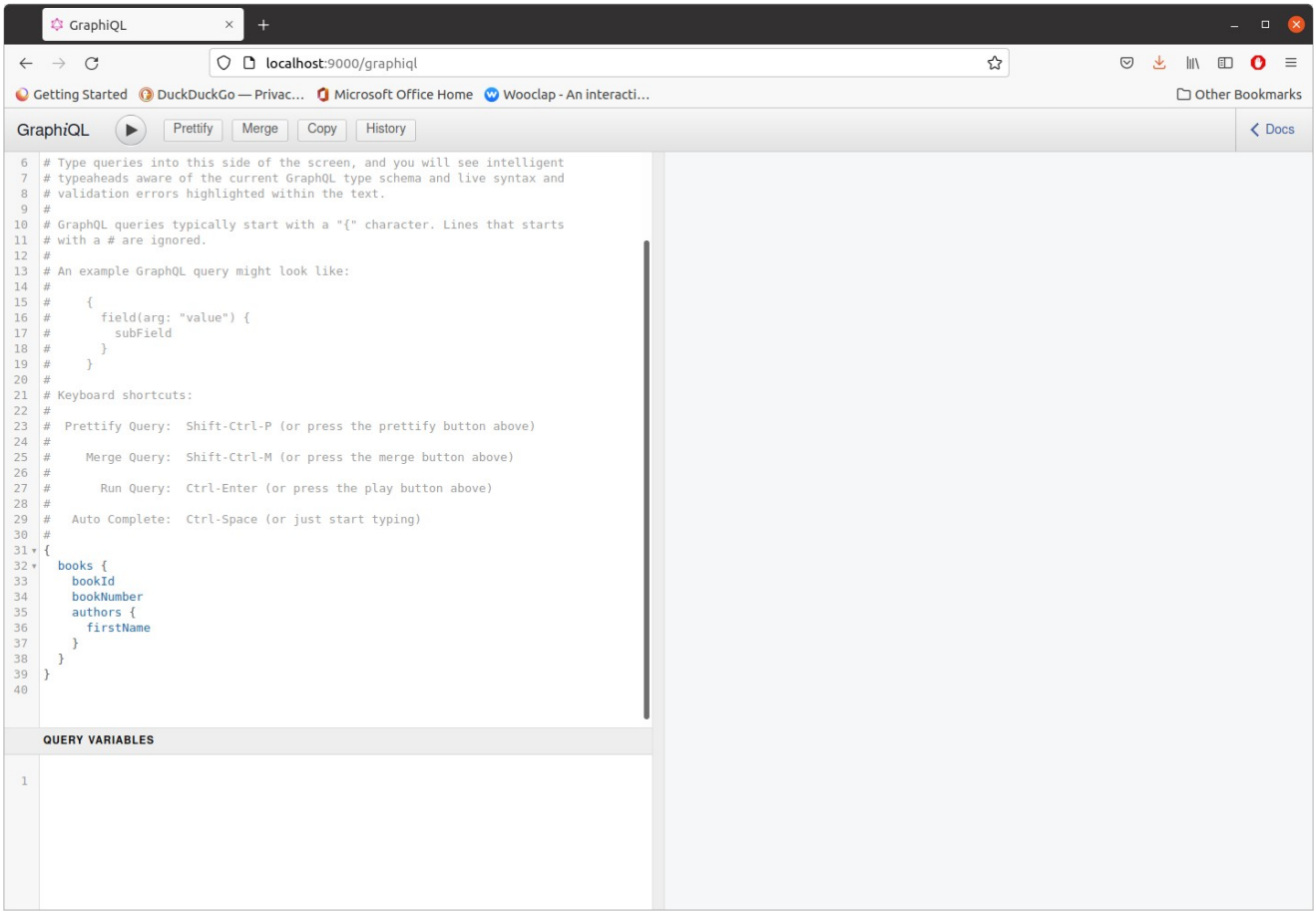
```
1.  package seg3x02.booksapigraphql.configuration
2.
3.  import org.springframework.context.annotation.Configuration
4.  import org.springframework.web.servlet.config.annotation.CorsRegistry
5.  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer
6.
7.  @Configuration
8.  class WebConfiguration : WebMvcConfigurer {
9.      override fun addCorsMappings(registry: CorsRegistry) {
10.         registry.addMapping("/**")
11.             .allowedOrigins("http://localhost:4200")
12.             .allowedMethods("*")
13.     }
14. }
```

Dans cette classe de configuration, nous spécifions des paramètres pour CORS en autorisant les requêtes Cross-Origin de notre application client Angular exécutant à l'adresse http://localhost:4200.

GraphiQL

GraphiQL est un outil qui communique avec les serveurs GraphQL et fournit une interface utilisateur pour exécuter des requêtes et des mutations. Nous pouvons inclure automatiquement une version Web de GraphiQL dans notre application, en activant la propriété graphql.graphiql.enabled comme cela a été fait précédemment.

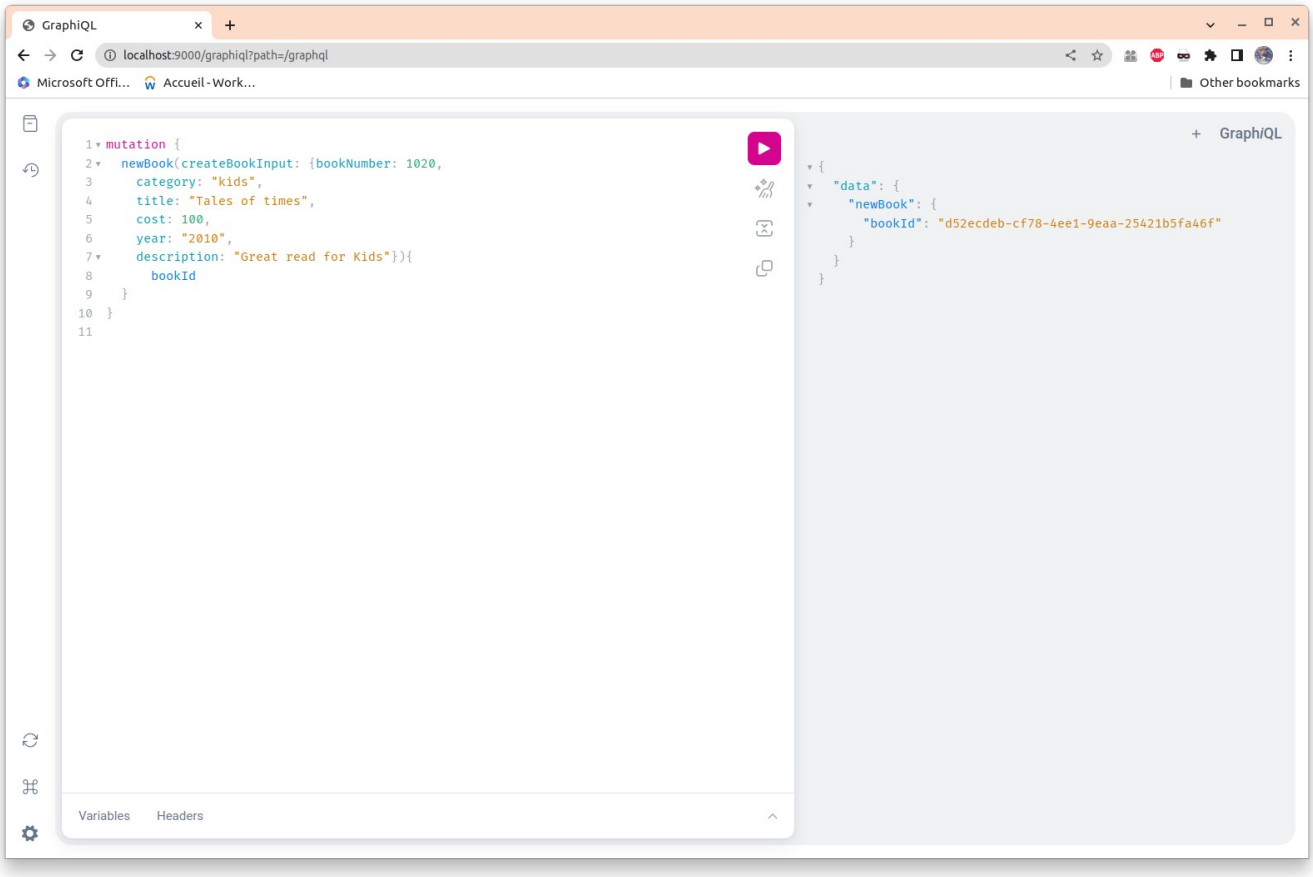
Une fois l'application en cours d'exécution, accédez à http://localhost:9000/graphiql pour lancer l'interface GraphiQL pour le serveur.



Exécutez la mutation suivante pour ajouter un livre.

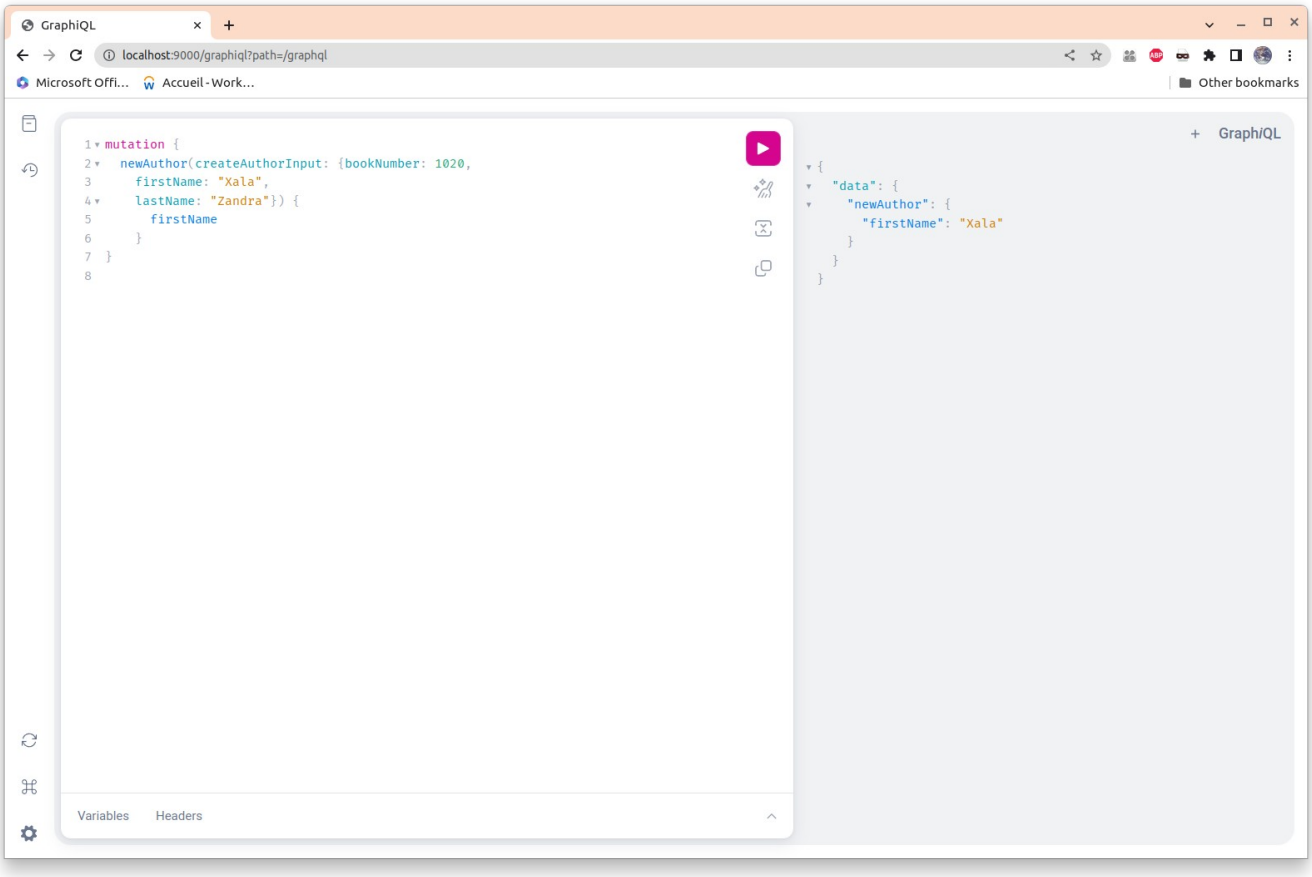
```
mutation {
 newBook(createBookInput: {bookNumber: 1020,
            category: "kids",
            title: "Tales of times",
            cost: 100,
            year: "2010",
            description: "Great read for Kids"}){
                  bookId
            }
}
```

La requête spécifie que le bookId du livre nouvellement créé soit retourné.



Nous exécutons à présent la mutation suivante pour ajouter un auteur au nouveau livre.

```
mutation {
  newAuthor(createAuthorInput: {bookNumber: 1020,
    firstName: "Xala",
    lastName: "Zandra"}) {
          firstName
    }
}
```



Voici une requête pour retourner tous les livres et pour chaque livre, uniquement le numéro de livre et le titre ainsi que les noms de famille des auteurs.

```
{
  books {
    bookNumber
    title
    authors {
      lastName
    }
  }
}
```

Client Angular

Nous allons mettre à jour l'exemple d'application Bookstore du Lab3 pour utiliser le serveur créé.

Setup

Nous utilisons Apollo (https://www.apollographql.com/) une bibliothèque qui fournit des outils pour interagir avec les serveurs GraphQL. Apollo Angular (https://apollo-angular.com/docs/) fournit un support spécifique à Angular.

Ajoutez Apollo Angular au projet: `ng add apollo-angular`

Spécifiez http://localhost:9000/graphql comme URL vers le point de terminaison GraphQL lorsque vous y êtes invité. Cela configure l'URL du serveur GraphQL dans le GraphQLModule dans `src/app/graphql.provider.ts`.

Modèle Book

Mettez à jour le modèle de Book (`src/app/books/model/bookEntity.ts`) comme suit.

```
1.  export class BookEntity {
2.    constructor(
3.      public bookId: number,
4.      public bookNumber: number,
5.      public category: string,
6.      public title: string,
7.      public cost: number,
8.      public authors?: AuthorEntity[],
9.      public year?: string,
10.     public description?: string
11.   ) {}
12. }
13.
14. export class AuthorEntity {
15.   constructor(
16.     public bookNumber: number,
17.     public firstName: string,
18.     public lastName: string
19.   ){}
20. }
```

Service Books

Mettez à jour le service Books (`src/app/books/service/books-service.ts`) en fonction du code source sur GitHub. Nous ne montrons ici qu'un extrait.

```
1.  import {inject, Injectable} from '@angular/core';
2.  import {AuthorEntity, BookEntity} from '../model/bookEntity';
```

```
3.  import {Apollo, gql} from "apollo-angular";
4.  import {Observable} from "rxjs";
5.  import {ApolloQueryResult, FetchResult} from "@apollo/client/core";
6.
7.  const GET_BOOKS= gql`
8.    query($bookNumber: Int!) {
9.      bookByNumber(bookNumber: $bookNumber) {
10.       bookId
11.       bookNumber
12.       category
13.       title
14.       cost
15.       description
16.       authors {
17.         firstName
18.         lastName
19.       }
20.     }
21.   }
22. `;
23.
24. const ADD_BOOK = gql`
25.   mutation newBook($bookNumber: Int!,
26.     $category: String!,
27.     $title: String!,
28.     $cost: Float!,
29.     $year: String,
30.     $description: String){
31.     newBook(createBookInput: {bookNumber: $bookNumber,
32.       category: $category,
33.       title: $title,
34.       cost: $cost,
35.       year: $year,
36.       description: $description}) {
37.       bookId
38.     }
39.   }
40. `;
41.
42. const ADD_AUTHOR = gql`
43.   mutation newAuthor($bookNumber: Int!,
44.     $firstName: String!,
45.     $lastName: String!){
46.     newAuthor(createAuthorInput: {bookNumber: $bookNumber,
47.       firstName: $firstName,
```

```
      lastName: $lastName}) {
      firstName
      lastName
    }
  }
`;

@Injectable({
  providedIn: 'root'
})
export class BooksService {
  private apollo: Apollo = inject(Apollo);

  public getBook(bookNumber: number): Observable<ApolloQueryResult<any>> {
    return this.apollo
    .query<any>({
      query: GET_BOOKS,
      variables: {
        bookNumber
      }
    });
  }

  public addBook(b: BookEntity): Observable<FetchResult<unknown>> {
    return this.apollo.mutate({
        mutation: ADD_BOOK,
        variables: {
          bookNumber: b.bookNumber,
          category: b.category,
          title: b.title,
          cost: b.cost,
          year: b.year,
          description: b.description
        }
      }
    );
  }

  addAuthor(author: AuthorEntity): Observable<FetchResult<unknown>> {
    return this.apollo.mutate({
        mutation: ADD_AUTHOR,
        variables: {
          bookNumber: author.bookNumber,
          firstName: author.firstName,
          lastName: author.lastName
```

```
93.         }
94.       }
95.     );
96.   }
97. }
```

Le service Books est désormais configuré pour utiliser le service Apollo (injecté à la ligne 59) pour interagir avec le serveur GraphQL. Le service Apollo prend les requêtes ou mutations GraphQL enveloppées dans la balise du modèle gql (https://github.com/apollographql/graphql-tag). Ces requêtes sont définies comme des constantes et envoyées au serveur à l'aide d'appolo. Les résultats sont retournés dans des Observables.

La fonction getBook (lignes 61-69) demande la recherche d'un livre correspondant à un numéro de livre donné, la fonction addBook (lignes 71-84) demande l'exécution d'une mutation pour ajouter un livre, et la fonction addAuthor (lignes 86-89) demande l'exécution d'une mutation pour ajouter un auteur de livre.

Composant Book

Mettez à jour la classe de composant de Book (`src/app/books/book/book.ts`) comme suit.

```
1.  import {Component, inject, OnDestroy, OnInit} from '@angular/core';
2.  import {ActivatedRoute} from '@angular/router';
3.  import {BookEntity} from '../model/bookEntity';
4.  import {BooksService} from '../service/books-service';
5.  import {Subscription} from "rxjs";
6.  import { AuthornamesPipe } from '../../pipes/authornames.pipe';
7.
8.  @Component({
9.    selector: 'app-book',
10.   templateUrl: './book.html',
11.   styleUrls: ['./book.css'],
12.   imports: [AuthornamesPipe]
13. })
14. export class Book implements OnInit, OnDestroy {
15.   selectedBook: BookEntity | null = null;
16.   private subscription!: Subscription;
17.   private route: ActivatedRoute = inject(ActivatedRoute);
18.   private booksService: BooksService = inject(BooksService);
19.
20.   ngOnInit(): void {
21.     this.route.params.subscribe(params => {
22.       const numb = Number(params['id']);
23.       this.subscription = this.booksService.getBook(numb).subscribe({
24.         next: ({data, loading}) => {
25.           this.selectedBook = data.bookByNumber;
```

```
26.         },
27.         error: (error: any) => {
28.           this.selectedBook = null;
29.         }
30.       });
31.     });
32.   }
33.
34.   ngOnDestroy(): void {
35.     this.subscription.unsubscribe();
36.   }
37. }
```

Nous souscrivons à l'Observable retourné par le Service Books (ligne 23) et récupérons les données retournées lorsqu'elles sont fournies.

Mettez à jour le modèle HTML du composant de Book comme suit.

```
1.  @if (selectedBook) {
2.    <div>
3.      <h2>Here are details of the Book {{selectedBook.bookNumber}}</h2>
4.      <div class="row">
5.        <div class="col-3">Category:</div>
6.        <div class="col-9">{{ selectedBook.category }}</div>
7.      </div>
8.      <div class="row">
9.        <div class="col-3">Title:</div>
10.       <div class="col-9">{{ selectedBook.title }}</div>
11.     </div>
12.     <div class="row">
13.       <div class="col-3">Cost:</div>
14.       <div class="col-9">{{ selectedBook.cost }}</div>
15.     </div>
16.     <div class="row" [hidden]="!selectedBook.authors">
17.       <div class="col-3">Author:</div>
18.       <div class="col-9"><span [innerHTML]="selectedBook.authors | authornames"></span></div>
19.     </div>
20.     <div class="row" [hidden]="!selectedBook.year">
21.       <div class="col-3">Year:</div>
22.       <div class="col-9">{{ selectedBook.year }}</div>
23.     </div>
24.     <div class="row" [hidden]="!selectedBook.description">
25.       <div class="col-3">Description:</div>
26.       <div class="col-9">{{ selectedBook.description }}</div>
27.     </div>
28.   </div>
29. }
```

```
@else {
  <div>
    <h2>Sorry can't find the requested book...</h2>
  </div>
}
```

Composant Admin

Mettez à jour la classe du composant Admin comme suit.

```
import {Component, inject} from '@angular/core';
import { AbstractControl, FormArray, FormBuilder, FormControl, Validators, FormsModule, ReactiveFormsModule } from '@angular/forms';
import {AuthorEntity, BookEntity} from '../books/model/bookEntity';
import {BooksService} from '../books/service/books-service';

function categoryValidator(control: FormControl<string>): { [s: string]: boolean } | null {
  const validCategories = ['Kids', 'Tech', 'Cook'];
  if (!validCategories.includes(control.value)) {
    return {invalidCategory: true};
  }
  return null;
}

@Component({
  selector: 'app-admin',
  templateUrl: './admin.html',
  styleUrls: ['./admin.css'],
  imports: [FormsModule, ReactiveFormsModule]
})
export class Admin {
  private builder: FormBuilder = inject(FormBuilder);
  private booksService: BooksService = inject(BooksService);
  bookForm = this.builder.group({
    bookNumber: ['', [Validators.required, Validators.pattern('[1-9]\\d{3}')]],
    category: ['', [Validators.required, categoryValidator]],
    title: ['', Validators.required],
    cost: ['', [Validators.required, Validators.pattern('\\d+(\\.\\d{1,2})?')] ],
    authors: this.builder.array([]),
    year: [''],
    description: ['']
  });

  get bookNumber(): AbstractControl {return <AbstractControl>this.bookForm.get('bookNumber'); }
  get category(): AbstractControl {return <AbstractControl>this.bookForm.get('category'); }
  get title(): AbstractControl {return <AbstractControl>this.bookForm.get('title'); }
  get cost(): AbstractControl {return <AbstractControl>this.bookForm.get('cost'); }
```

```
37.   get authors(): FormArray {
38.     return this.bookForm.get('authors') as FormArray;
39.   }
40.
41.   onSubmit(): void {
42.     if (this.bookForm.valid) {
43.       const book = new BookEntity(0,
44.         Number(this.bookForm.value.bookNumber),
45.         <string>this.bookForm.value.category,
46.         <string>this.bookForm.value.title,
47.         Number(this.bookForm.value.cost),
48.         [],
49.         <string>this.bookForm.value.year,
50.         <string>this.bookForm.value.description);
51.       const authors = <AuthorEntity[]>this.bookForm.value.authors;
52.       this.booksService.addBook(book).subscribe(_ => {
53.         authors.forEach((author: AuthorEntity) => {
54.           this.booksService.addAuthor(Object.assign(author, {bookNumber: book.bookNumber}))
55.             .subscribe(_ => {
56.             });
57.         });
58.       });
59.       this.bookForm.reset();
60.       this.authors.clear();
61.     }
62.   }
63.
64.   addAuthor(): void {
65.     this.authors.push(
66.       this.builder.group({
67.         firstName: [''],
68.         lastName: ['']
69.       })
70.     );
71.   }
72.
73.   removeAuthor(i: number): void {
74.     this.authors.removeAt(i);
75.   }
76. }
```

Le principal changement concerne la fonction onSubmit où le service de livres est utilisé pour ajouter un livre (ligne 52). On s'abonne alors à l'Observable qui est retourné et une fois terminé, on invoque le Books Service pour ajouter tous les auteurs du nouveau livre (lignes 53-56).

Mettez à jour le modèle HTML du composant Admin comme suit.

```
<div class="container">
  <h1>BookEntity Form</h1>
  <button (click)="addAuthor()" class="btn btn-primary">Add Author</button>
  <form [formGroup]="bookForm" (ngSubmit)="onSubmit()">
    <div class="mb-3">
      <label for="bookNumber">Number:</label>
      <input type="text" class="form-control" id="bookNumber"  formControlName="bookNumber" required>
      @if (bookNumber.dirty && !bookNumber.valid) {
        <div class="alert alert-danger">
          BookEntity number must be 4 digits long starting with a number different from 0.
        </div>
      }
    </div>
    <div class="mb-3">
      <label for="category">Category:</label>
      <input type="text" class="form-control" id="category" formControlName="category">
      @if (category.dirty && !category.valid) {
        <div class="alert alert-danger">
          Category is required to be <b>Kids</b>, <b>Tech</b> or <b>Cook</b>
        </div>
      }
    </div>
    <div class="mb-3">
      <label for="title">Title:</label>
      <input type="text" class="form-control" id="title" required formControlName="title">
      @if (title.dirty && !title.valid) {
        <div class="alert alert-danger">
          Title is required.
        </div>
      }
    </div>
    <div class="mb-3">
      <label for="cost">Cost:</label>
      <input type="text" class="form-control" id="cost" required pattern="\d+(.\d{1,2})?" formControlName="cost">
      @if (cost.dirty && !cost.valid) {
        <div class="alert alert-danger">
          Cost should be a number with two optional decimals
        </div>
      }
    </div>
    <div formArrayName="authors">
      @for (_ of authors.controls; track $index) {
```

```
43.         <div>
44.           <ng-container [formGroupName]="$index">
45.             <div class="row g-lg-5 align-items-center">
46.               <div class="col-auto">
47.                 <label for="authName-{{$index}}" class="col-form-label">Author First Name:</label>
48.               </div>
49.               <div class="col-auto">
50.                 <input id="authName-{{$index}}" class="form-control" formControlName="firstName"
    type="text">
51.               </div>
52.             </div>
53.             <div class="row g-lg-5 align-items-center">
54.               <div class="col-auto">
55.                 <label for="authLName-{{$index}}" class="col-form-label">Author Last Name:</label>
56.               </div>
57.               <div class="col-auto">
58.                 <input id="authLName-{{$index}}" class="form-control" formControlName="lastName"
    type="text">
59.               </div>
60.               <div class="col-auto">
61.                 <button class="btn btn-dark form-control" (click)="removeAuthor($index)">X</button>
62.               </div>
63.             </div>
64.           </ng-container>
65.         </div>
66.       }
67.     </div>
68.     <div class="mb-3">
69.       <label for="year">Year:</label>
70.       <input type="text" class="form-control" id="year" formControlName="year">
71.     </div>
72.     <div class="mb-3">
73.       <label for="description">Description:</label>
74.       <textarea cols="40" class="form-control" id="description"
    formControlName="description"></textarea>
75.     </div>
76.     <button type="submit" class="btn btn-success" [disabled]="bookForm.invalid">Submit</button>
77.   </form>
78. </div>
```

Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Lorsque vous accepterez le devoir pour ce lab sur Github, votre référentiel sera initialisé par du code d'une application de démarrage. Le référentiel Github de démarrage contient dans la branche *client*, l'application de saisie de données d'employés de l'exercice du laboratoire 5, modifiée pour conserver les données dans un serveur GraphQL.

Il vous est demandé de terminer l'implémentation du serveur GraphQL dans la branche *server*, en ajoutant un schéma GraphGL (*employee.graphqls*) et du code pour les résolveurs (*EmployeesResolver*). Notez qu'il est uniquement requis d'ajouter suffisamment d'implémentation pour que le serveur résultant puisse fonctionner avec le client Angular fourni.